

Architecture Decision Record

Quick Start Pro — Driving School Management System

COE 454 — Software Engineering II

Team Xenon

Week 3 Deliverable | COE 454 Client Project

Date: 22 July 2026

Client	Quick Start Pro Driving School
Client Contact	Reagan Masud (Owner) · Mercy Otoo (Secretary)
Project	Driving School Management System (Web Application)
Document Owner	Team Xenon
ADR Count	6 (Minimum required: 5)
Status	All ADRs — Accepted

Table of Contents

- ADR-001 — Frontend Framework and Tooling
- ADR-002 — Backend Runtime and Framework
- ADR-003 — Database Selection and Hosting
- ADR-004 — Authentication and Authorisation Approach
- ADR-005 — Deployment Platform
- ADR-006 — API Design Style

Status: Accepted

CONTEXT

The project requires a responsive, modern web application for a driving school with two distinct user roles — Secretary and Admin. The interface must be developed collaboratively by multiple frontend developers within a six-week academic timeline. The client's secretary (Mercy Otoo) and the administrator (Reagan Masud) both need a clean, usable dashboard accessible on desktop browsers. The team needed a frontend framework that offers strong component reusability, excellent documentation, TypeScript support, and fast local development cycles.

DECISION

The team will use React 19 as the UI library, TypeScript for type safety, Vite as the build tool and dev server, and Tailwind CSS for styling. React Router will handle client-side navigation, React Hook Form with Zod will manage form validation, Axios will handle API communication, and Lucide React will provide the icon set. The application will follow a single shared layout shell with role-based rendering to serve both the Secretary and Admin without duplicating the interface.

CONSEQUENCES

Advantages:

- React's component model enables reuse across the Secretary and Admin views, reducing development effort significantly.
- TypeScript catches type mismatches at compile time, reducing runtime bugs in form handling and API responses.
- Vite provides near-instant HMR (Hot Module Replacement), enabling fast iterative UI development.
- Tailwind CSS eliminates context-switching between stylesheet files and keeps styling co-located with components.
- The ecosystem is large and well-documented, lowering the learning curve for all team members.
- React Hook Form with Zod provides clean, validated multi-step forms required for student registration.

Disadvantages:

- Team members unfamiliar with TypeScript face an initial learning curve configuring types for API responses and component props.
- Global state management (e.g., for auth context and user roles) requires careful design to avoid prop-drilling across nested components.
- Tailwind's utility-class approach can make component files verbose without consistent component abstraction patterns.

Status: Accepted

CONTEXT

The application requires a server-side component to expose REST API endpoints for student management, scheduling, payments, attendance, and reporting. The backend must support JWT-based authentication, role-based access control, and interact with a relational database. The team needed a backend stack that is easy to develop quickly, shares the same language as the frontend (reducing cognitive overhead), and is well-supported for deployment on free-tier cloud platforms within the project timeline.

DECISION

The team will use Node.js as the runtime and Express.js as the HTTP framework, written in TypeScript. The project will follow a structured layered architecture: routes → controllers → services → database queries. The `tsx` package (not `ts-node`) is used to run TypeScript directly in development, as it is compatible with TypeScript 7. The backend will expose a RESTful API consumed by the React frontend via `Axios`.

CONSEQUENCES

Advantages:

- Node.js and Express share JavaScript/TypeScript with the frontend, eliminating the language-switch cost and enabling shared type definitions.
- Express is minimalist and flexible, allowing the team to structure the project exactly as needed without framework constraints.
- The layered architecture (routes → controllers → services) keeps concerns separated and makes the codebase easier to navigate and test.
- Large community support means abundant tutorials, middleware packages, and debugging resources.
- `tsx` (used in place of `ts-node-dev`) is fully compatible with TypeScript 7, avoiding compatibility errors encountered with older tools.
- Deployment to Railway or Render is straightforward with a simple start script.

Disadvantages:

- Express provides no built-in structure, so discipline is required to maintain consistent patterns as the codebase grows.
- Node.js is single-threaded; CPU-intensive operations (e.g., PDF report generation) could block the event loop if not handled asynchronously.
- Error handling middleware must be explicitly implemented — Express does not provide standardized error responses out of the box.

Status: Accepted

CONTEXT

The application manages highly relational data: students belong to packages, lessons are scheduled for students with specific instructors at specific times, payments reference students and are recorded by staff, and attendance links students to lesson sessions. The database needed to enforce referential integrity, support complex queries (e.g., outstanding balances, attendance history), and be hosted on a platform accessible to the whole team during development. The team also needed to avoid database hosting costs within the academic timeline.

DECISION

The team will use PostgreSQL as the primary relational database. It will be hosted on Supabase, which provides a managed PostgreSQL instance with a generous free tier, a web-based SQL editor, and a direct connection string compatible with the Node.js pg driver. The team will use plain SQL queries via the pg library (not an ORM) for initial development, keeping the database interaction layer simple and auditable. Schema changes will be managed through versioned migration scripts.

CONSEQUENCES

Advantages:

- PostgreSQL's support for foreign keys, constraints, and joins is essential for this project's relational data model (students, lessons, payments, attendance).
- ACID compliance ensures that payment recording and attendance marking are transactionally safe — a critical requirement for a financial workflow.
- Supabase provides a zero-configuration managed PostgreSQL instance with a visual table editor, useful for quickly inspecting data during development.
- The Supabase connection string works directly with the pg library, requiring no additional configuration.
- Free tier is sufficient for the expected data volume of an MVP serving one driving school.
- PostgreSQL's JSON column support provides flexibility if semi-structured data is needed in future phases.

Disadvantages:

- The team must write and manage SQL migrations manually, as no ORM migration tool is being used in this phase.
- Team members unfamiliar with SQL may require time to become productive writing JOIN-heavy queries for reports.
- Supabase free tier has connection limits and compute constraints that may require upgrading in Phase 2 (Weeks 7–9) during load testing.

Status: Accepted

CONTEXT

The Quick Start Pro system is an internal staff tool. Only two user roles exist: Secretary (Mercy Otoo) and Admin (Reagan Masud, the business owner). There is no self-registration flow — accounts are created by the Admin. The system does not require social login, multi-factor authentication, or integration with an external identity provider. The team needed a lightweight, stateless authentication mechanism that is straightforward to implement, test, and understand within the academic timeline.

DECISION

The team will implement JWT (JSON Web Token) authentication. On successful login, the server issues a signed access token containing the user's ID and role. This token is stored in the client's memory (React context) and sent as a Bearer token in the Authorization header on every API request. Passwords are hashed with bcrypt (salt rounds: 12) before storage. Role-based access control (RBAC) is enforced via Express middleware that decodes the token and checks the role before allowing access to protected routes. The Admin role has access to all routes; the Secretary role has access to a defined subset.

CONSEQUENCES

Advantages:

- JWT is stateless — the server does not need to maintain a session store, simplifying the backend and enabling horizontal scaling if needed.
- bcrypt with 12 salt rounds is an industry-standard approach to password hashing that is resistant to brute-force attacks.
- Role information encoded in the token allows the frontend to render role-appropriate UI without an additional API call.
- The implementation requires no third-party authentication service, keeping the system self-contained and reducing dependencies.
- Straightforward to test: authentication middleware can be unit-tested by mocking token payloads.

Disadvantages:

- Access tokens cannot be individually revoked before expiry without implementing a token blocklist, which adds complexity. Mitigation: use short expiry times (e.g., 15 minutes) with refresh tokens, or accept the limitation given the internal-only use case.
- Storing tokens in React context (memory) means users are logged out on page refresh unless a refresh token strategy or localStorage persistence is added. The team will implement localStorage persistence with appropriate XSS precautions.
- If the JWT_SECRET is compromised, all issued tokens are invalidated — secret rotation must be handled carefully in the deployment environment.

Status: Accepted

CONTEXT

The MVP must be publicly accessible by Week 5 for the client demo (Site Visit 3). The team required hosting platforms that are free or low-cost for students, support automatic deployment from GitHub, provide HTTPS by default, and require minimal DevOps configuration. The frontend and backend are separate applications (decoupled architecture) and therefore require separate hosting solutions.

DECISION

The frontend (React/Vite) will be deployed to Vercel. The backend (Node.js/Express) will be deployed to Railway. The database (PostgreSQL) is hosted on Supabase as decided in ADR-003. Vercel is connected to the GitHub repository's main branch and triggers automatic deployments on every merge. Railway is similarly configured via its GitHub integration. Environment variables for all services are managed through each platform's secrets dashboard; no secrets are committed to the repository.

CONSEQUENCES

Advantages:

- Vercel is purpose-built for React/Vite applications and produces optimized builds with a global CDN automatically.
- Railway supports Node.js backends with zero Dockerfile configuration, detecting the runtime and start command automatically.
- Both platforms integrate directly with GitHub, enabling the team's pull-request workflow to automatically produce preview deployments.
- HTTPS is enabled by default on both platforms with no additional configuration.
- Free tiers on Vercel and Railway are sufficient for an MVP serving a single client with low concurrent usage.
- The three-platform split (Vercel + Railway + Supabase) follows the principle of using purpose-built tools for each layer.

Disadvantages:

- Railway free tier applications may be suspended after a period of inactivity, adding a cold-start delay for the first request — relevant to the client demo scenario.
- Free-tier resource limits (CPU, memory, bandwidth) may be reached during the Week 7 load test and will need to be accounted for in the Scalability Gap Analysis.
- Managing environment variables across three platforms increases the risk of configuration drift; the team must document all required variables in .env.example files.

Status: Accepted

CONTEXT

The frontend and backend are decoupled applications that communicate over HTTP. The team needed to choose a communication pattern that is well-understood by all developers, easy to document (as required by the Week 3 API Schema deliverable), and straightforward to test with standard tools. The alternatives considered were REST (Representational State Transfer) and GraphQL.

DECISION

The team will design and implement a RESTful API. Resources are organized around the core domain entities: /students, /lessons, /payments, /attendance, /staff, /auth, and /reports. Standard HTTP verbs (GET, POST, PUT, PATCH, DELETE) are used to express operations. All responses return JSON. API versioning is prefixed as /api/v1/. Error responses follow a consistent shape: { error: true, message: string, code: string }. The API Schema document (a separate Week 3 deliverable) lists all planned endpoints with method, path, request body, and response shape.

CONSEQUENCES

Advantages:

- REST is universally understood — every team member can read, write, and test REST endpoints without learning a new query language.
- Standard HTTP tooling (Postman, curl, browser DevTools) works natively with REST, simplifying debugging and testing.
- The Week 3 deliverable requires an API Schema document listing all endpoints: REST's resource-based structure makes this straightforward to produce.
- RESTful conventions are directly mappable to Express router definitions, keeping the codebase predictable.
- Axios on the frontend handles REST requests with minimal configuration and supports request interceptors for attaching JWT tokens.

Disadvantages:

- Some dashboard views (e.g., the Admin summary combining revenue, attendance, and student counts) may require multiple API calls, increasing network overhead compared to a single GraphQL query.
- As the feature set grows, the number of endpoints increases proportionally — disciplined URL naming and versioning will be required to prevent route sprawl.